Database development with PL/SQL INSY 8311

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | eric.maniraguha@auca.ac.rw | LinkedIn Profile

6h00 pm – 9h50 pm

- Monday A -G207
- Tuesday B-G204
- Wednesday E-106
- Thursday F-G307

March 2024

Database development with PL/SQL

Reference reading

- SQL Function
- PL/SQL Exception Handling
- CREATE FUNCTION Statement
- What is Package in Oracle?
- Pipelined Table Functions
- Pipelined Table Functions
- What is PL/SQL? Explained
- SQL CASE Statement with Multiple Conditions
- Database Security
- 7 Coding PL/SQL Procedures and Packages

Lecture 06 – Functions

Objectives of Function

Understand PL/SQL Functions:

- Learn the differences between procedures and functions in PL/SQL.
- Understand when and how to use functions effectively in various scenarios.

Design and Implement Functions:

- Comprehend the structure and purpose of functions in PL/SQL.
- Learn how to create functions that return computed values.
- Explore integrating functions into SQL queries and PL/SQL blocks.

Execute and Debug PL/SQL Functions:

- Gain hands-on experience in executing functions.
- Practice debugging techniques to ensure correct function execution and error handling.

Key Points about Procedures in PL/SQL

A **procedure** in PL/SQL is a subprogram that performs operations such as modifying database records, calculations, or calling other subprograms. It has:

- A **specification** (using the PROCEDURE keyword)
- A **body** (starting with IS or AS)

Parameters are optional and can be included in the specification.

Procedure Body:

1. **Declarative Part** (Optional): Declares variables, constants, exceptions, and types.
2. **Executable Part**: Contains the executable statements.
3. **Exception-Handling Part** (Optional): Handles exceptions.

Parameter Modes:

- **IN**: Passes values to the procedure (treated as constants).
- **OUT**: Returns values to the caller.
- **IN OUT**: Passes initial values and returns updated ones.

Parameter Initialization:

- IN parameters can be initialized using := or DEFAULT.

Autonomous Transactions:

A procedure can be **autonomous** using PRAGMA AUTONOMOUS_TRANSACTION, allowing independent transactions from the main one.

AUTHID Clause:

Specifies whether the procedure runs with the owner's privileges or the current user's privileges.

Calling Procedures:

- Procedures are invoked as simple PL/SQL statements, with arguments passed using either positional or named notation.

PL/SQL Function Syntax

Function Syntax without Exception Handling

```
CREATE OR REPLACE FUNCTION function_name (
    parameter_name datatype  -- Input parameter for the function
)
RETURN return_datatype  -- Data type of the value the function will return
AS
    -- Local variable declarations (optional)
    -- local_variable datatype;
BEGIN
    -- Function logic
    -- Example: Perform some calculation or operation
    -- local_variable := some_operation(parameter_name);

    -- Compute and return the result
    RETURN result_value;  -- Return the final computed result

    -- Comments explaining the function logic can be added here for clarification
END;
/
```

Function Syntax with Exception Handling

```
CREATE [OR REPLACE] FUNCTION function_name
    [ (parameter1 datatype, parameter2 datatype, ...) ]
RETURN return_type
IS
    -- Declare local variables, constants, types, and exceptions (optional)
BEGIN
    -- Function logic and computations
    -- Return the result
    RETURN return_value;
EXCEPTION
    -- Exception handling (optional)
    WHEN exception_name THEN
        -- Handle the exception
END function_name;
```

Key Elements:

- **CREATE [OR REPLACE] FUNCTION**: Defines the function. OR REPLACE is optional and allows updating an existing function.
- **function_name**: The name of the function.
- **parameter1, parameter2, ...**: Optional parameters that the function accepts.
- **RETURN return_type**: Specifies the data type that the function will return (e.g., NUMBER, VARCHAR2, etc.).
- **IS/AS**: Marks the beginning of the function body.
- **BEGIN/END**: Encapsulates the function's logic, where the computation happens.
- **RETURN return_value**: The value to be returned from the function.
- **EXCEPTION**: (Optional) Handles exceptions raised during the function execution.

Procedure Vs. Function: Key Differences

Feature	Function	Procedure
Return Value	Always returns a value (scalar, table, or collection).	Does not return a value but can use OUT/IN OUT parameters.
Usage	Used for computations and returning values in SQL (SELECT, WHERE, JOIN).	Used for business logic, executing actions, and modifying data.
Syntax	Includes a RETURN clause specifying the return type.	No RETURN clause required.
Calling	Can be called in SQL statements (SELECT, WHERE, JOIN) and PL/SQL blocks.	Called using the CALL statement in PL/SQL blocks.
Parameters	Only IN parameters.	Supports IN, OUT, and IN OUT parameters.
Purpose	Performs calculations and returns a value.	Executes actions, modifies data, and returns multiple values via OUT parameters.
Example	CREATE FUNCTION get_employee_name(emp_id NUMBER) RETURN VARCHAR2 AS ... END;	CREATE PROCEDURE update_employee_salary(emp_id NUMBER, new_salary NUMBER) AS ... END;
When to Use	Use for computations and returning values, often inside SQL queries.	Use for executing actions, modifying data, and returning multiple values.

Similarities between Procedure and Function	Both can be called from other PL/SQL blocks.
	If the exception raised in the subprogram is not handled in the subprogram exception handling section, then it will propagate to the calling block.
	Both can have as many parameters as required.
	Both are treated as database objects in PL/SQL.

Functions vs. Procedures: Advantages of Using Functions

In PL/SQL, **functions** are often preferred over **procedures** in certain scenarios due to their specific advantages. Here's why:

1. Return Values

- **Function:** Always returns a value (scalar, record, or table).
- **Procedure:** Does not return a value directly; instead, it modifies data via OUT parameters.

✅ **Advantage:** Functions make it easier to use in expressions (e.g., SELECT my_function FROM dual).

2. Use in SQL Statements

- **Function:** Can be used in SQL queries (e.g., SELECT my_function(col) FROM table).
- **Procedure:** Cannot be used in SQL queries.

✅ **Advantage:** Functions allow modular and reusable logic within SQL.

3. Deterministic Behavior

- **Function:** Can be declared as **DETERMINISTIC**, meaning it returns the same result for the same input, improving query performance via result caching.
- **Procedure:** Cannot be deterministic.

✅ **Advantage:** Optimized performance when used in queries.

4. Better Readability and Maintainability

- **Function:** Encapsulates logic into a single-returning unit, making the code easier to read.
- **Procedure:** May require multiple OUT parameters, making it harder to track results.

✅ **Advantage:** Functions promote cleaner code.

5. Encapsulation in PL/SQL Expressions

- **Function:** Can be used inside expressions (e.g., IF my_function(x) = 1 THEN ...).
- **Procedure:** Cannot be used in expressions.

✅ **Advantage:** Functions are more flexible in PL/SQL programming.

PL/SQL Procedure vs Function

Procedure

```
-- Procedure to update the salary of an employee based on their employee ID
CREATE OR REPLACE PROCEDURE update_salary (
    p_emp_id      IN  employees.employee_id%TYPE,  -- Input parameter: employee ID
    p_new_salary  IN  employees.salary%TYPE        -- Input parameter: new salary
) IS
BEGIN
    -- Update the salary for the given employee ID
    UPDATE employees
    SET salary = p_new_salary
    WHERE employee_id = p_emp_id;
END;
/
```

```
-- Call the procedure to update salary
BEGIN
    update_salary(101, 5000);  -- Update employee 101's salary to 5000
END;
/
```

Function

```
-- Function to get the full name of an employee based on their employee ID
CREATE OR REPLACE FUNCTION get_full_name (
    p_emp_id IN employees.employee_id%TYPE  -- Input parameter: employee ID
) RETURN VARCHAR2 - No length needed
IS
    v_full_name (100) -- Declare variable to store full name
BEGIN
    -- Fetch the full name from the employees table
    SELECT first_name || ' ' || last_name
    INTO v_full_name
    FROM employees
    WHERE employee_id = p_emp_id;

    RETURN v_full_name;  -- Return the full name
END;
/
```

```
DECLARE
    v_emp_id     employees.employee_id%TYPE := 101;  -- Replace with actual employee ID
    v_full_name  VARCHAR2(100);
BEGIN
    -- Call the function
    v_full_name := get_full_name(v_emp_id);

    -- Display the full name
    DBMS_OUTPUT.PUT_LINE('Full Name: ' || v_full_name);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Employee with ID ' || v_emp_id || ' not found.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
/
```

PL/SQL Function – Example I

```
1.  CREATE OR REPLACE FUNCTION welcome_msg_func ( p_name IN VARCHAR2)
2.  RETURN VARCHAR2
3.  IS
4.  BEGIN
5.  RETURN ('Welcome '|| p_name);
6.  END;
7.  /
```

Output:

Function created

Function Created

```
8.  DECLARE
9.  lv_msg VARCHAR2(250);
10. BEGIN
11. lv_msg := welcome_msg_func ('Guru99');
12. dbms_output.put_line(lv_msg);
13. END;
```

Calling function with 'Guru99' as parameter

Output:

Welcome Guru99

```
14. SELECT welcome_msg_func('Guru99') FROM DUAL;
```

Output:

Welcome Guru99

In PL/SQL, DUAL is a special, one-row, one-column table that is commonly used for selecting a value or expression without needing to reference a real table

Key Points About DUAL:

1. **Purpose**: DUAL is used to select a constant value, perform calculations, or call functions without needing to reference a real table.
2. **Structure**: The DUAL table has only one row and one column, with the column named DUMMY (of type VARCHAR2) and the value in that column is always 'X'.

```
SET SERVEROUTPUT ON;

BEGIN
  -- Example of calling a function in PL/SQL
  FOR i IN 1..5 LOOP
    -- Using DUAL to calculate square of i
    DBMS_OUTPUT.PUT_LINE('Square of ' || i || ' is ' || (i * i));
  END LOOP;
END;
```

Output

```
Square of 1 is 1
Square of 2 is 4
Square of 3 is 9
Square of 4 is 16
Square of 5 is 25
```

Source Image: https://www.guru99.com/subprograms-procedures-functions-pl-sql.html

PL/SQL Deterministic Function – Example

```
CREATE OR REPLACE FUNCTION get_discount(price NUMBER)
RETURN NUMBER
DETERMINISTIC
IS
BEGIN
   RETURN price * 0.1;  -- Returns the same result for the same input
END;
/
```

- **DETERMINISTIC**: This keyword indicates that the function will always return the same result for the same input (i.e., the discount for a given price will always be the same).
- **price NUMBER**: This is the input parameter.
- **RETURN NUMBER**: The function returns a number (the calculated discount).

Calling in a SELECT Statement

```
SELECT get_discount(100) AS discount
FROM DUAL;
```

Calling in a PL/SQL Block

```
DECLARE
   v_price NUMBER := 200;
   v_discount NUMBER;
BEGIN
   v_discount := get_discount(v_price);
   DBMS_OUTPUT.PUT_LINE('Discount on price ' || v_price || ' is: ' || v_discount);
END;
/
```

PL/SQL Function with Exception Handling

Function created

```
CREATE OR REPLACE FUNCTION calculate_area(radius IN NUMBER)
RETURN NUMBER
IS
    area NUMBER;
BEGIN
    -- Check for invalid radius value (negative or zero)
    IF radius <= 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Radius must be greater than zero');
    END IF;

    -- Calculate the area
    area := 3.1416 * radius * radius;

    -- Return the calculated area
    RETURN area;

EXCEPTION
    WHEN OTHERS THEN
        -- In case of an exception, log the error message and return NULL
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);  -- Display the error message
        RETURN NULL;
END calculate_area;
/
```

Calling the function

```
SET SERVEROUTPUT ON;

DECLARE
    v_area NUMBER;
BEGIN
    -- Calling the function with an invalid radius value (negative)
    v_area := calculate_area(5);  -- Example with radius = -5 (invalid)

    -- Display the calculated area only if the value is not NULL
    IF v_area IS NOT NULL THEN
        DBMS_OUTPUT.PUT_LINE('Calculated Area: ' || v_area);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Invalid radius value or error
occurred.');
    END IF;

END;
/
```

```
Function CALCULATE_AREA compiled

Calculated Area: 78.54

PL/SQL procedure successfully completed.
```

PLSQL Function Example II

Anonymous Function

```
DECLARE
    -- Variables to hold input numbers and the result
    a NUMBER := 23;
    b NUMBER := 45;
    c NUMBER;

    -- Function to find the maximum of two numbers
    FUNCTION findMax(x IN NUMBER, y IN NUMBER) RETURN NUMBER IS
        z NUMBER;  -- Local variable to store the maximum
    BEGIN
        -- Determine the greater of the two values
        IF x > y THEN
            z := x;
        ELSE
            z := y;
        END IF;

        RETURN z;  -- Return the maximum value
    END findMax;

BEGIN
    -- Call the function and assign the result to c
    c := findMax(a, b);

    -- Display the result
    DBMS_OUTPUT.PUT_LINE('Maximum of (' || a || ', ' || b || '): ' || c);
END;
/
```

Optional: Parameterize Function


```
-- Create or replace a stored function named 'findMax'
-- This function takes two NUMBER inputs and returns the greater of the two
CREATE OR REPLACE FUNCTION findMax(
    x IN NUMBER,  -- First input number
    y IN NUMBER   -- Second input number
)
RETURN NUMBER  -- The function returns a NUMBER
IS
BEGIN
    -- Use a CASE expression to return the greater of the two numbers
    RETURN CASE
            WHEN x > y THEN x  -- If x is greater than y, return x
            ELSE y             -- Otherwise, return y
           END;
END;
/
```

Calling the function

```
DECLARE
    a NUMBER := 34;
    b NUMBER := 12;
    c NUMBER;
BEGIN
    c := findMax(a, b);
    DBMS_OUTPUT.PUT_LINE('Max is: ' || c);
END;
/
```

Call a function inside another function or Pass a function as a parameter

Calling a Function Inside Another Function

```
DECLARE
  -- Declare a variable to store the result
  v_result NUMBER;

  -- Function to calculate square
  FUNCTION get_square(p_number NUMBER) RETURN NUMBER IS
  BEGIN
    RETURN p_number * p_number;
  END get_square;

  -- Function to calculate cube using the square function
  FUNCTION get_cube(p_number NUMBER) RETURN NUMBER IS
  BEGIN
    RETURN p_number * get_square(p_number);
  END get_cube;

BEGIN
  -- Call the function get_cube with 3 as input
  v_result := get_cube(3);

  -- Output the result
  DBMS_OUTPUT.PUT_LINE('Cube of 3 is: ' || v_result);
END;
/
```

Output

```
Cube of 3 is: 27
```

Using a Function Call as a Parameter in Another Function

```
DECLARE
  -- Declare a variable to store the result
  v_result NUMBER;

  -- Function to return the square of a number
  FUNCTION get_square(p_number NUMBER) RETURN NUMBER IS
  BEGIN
    RETURN p_number * p_number;
  END get_square;

  -- Function to add 10 to a given number
  FUNCTION add_ten(p_number NUMBER) RETURN NUMBER IS
  BEGIN
    RETURN p_number + 10;
  END add_ten;

BEGIN
  -- Pass function call as a parameter
  v_result := add_ten(get_square(4));  -- Equivalent to add_ten(16)

  -- Output the result
  DBMS_OUTPUT.PUT_LINE('Result of add_ten(get_square(4)) is: ' || v_result);
END;
/
```

Output

```
Result of add_ten(get_square(4)) is: 26
```

Using a Function Inside a Procedure

Calling a Function Inside Another Function

```
/*
1. A function (get_square) that returns the square of a number.
2. A procedure (display_square) that calls the function and prints the result.

*/

DECLARE
  -- Function to calculate square of a number
  FUNCTION get_square(p_number NUMBER) RETURN NUMBER IS
  BEGIN
    RETURN p_number * p_number;
  END get_square;

  -- Procedure that calls the function and displays the result
  PROCEDURE display_square(p_number NUMBER) IS
    v_result NUMBER;
  BEGIN
    -- Call the function inside the procedure
    v_result := get_square(p_number);

    -- Display the result
    DBMS_OUTPUT.PUT_LINE('Square of ' || p_number || ' is: ' || v_result);
  END display_square;

BEGIN
  -- Call the procedure
  display_square(5); -- Passing 5 as input
END;
/
```

Output

```
Square of 5 is: 25
```

- **Step 1: Create get_salary(emp_id) Function**

```
CREATE OR REPLACE FUNCTION get_salary(emp_id
NUMBER) RETURN NUMBER IS
    v_salary NUMBER;
BEGIN
    -- Fetch salary from the employees table
    SELECT salary INTO v_salary FROM employees WHERE
employee_id = emp_id;
    RETURN v_salary;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;  -- Return 0 if no employee found
END get_salary;
/
```

Call function

```
SELECT get_salary(101) FROM dual;
```

- **Step 2: Create calculate_bonus(emp_id) Function**

```
CREATE OR REPLACE FUNCTION calculate_bonus(emp_id
NUMBER) RETURN NUMBER IS
    v_salary NUMBER;
    v_bonus NUMBER;
BEGIN
    v_salary := get_salary(emp_id);  -- Calling get_salary
function
    v_bonus := v_salary * 0.1;  -- Bonus is 10% of salary
    RETURN v_bonus;
END calculate_bonus;
/
```

Call function

```
SELECT calculate_bonus(101) FROM dual;
```

- **Step 3: Create total_compensation(emp_id) Function**

```
CREATE OR REPLACE FUNCTION total_compensation(emp_id NUMBER) RETURN NUMBER IS
    v_salary NUMBER;
    v_bonus NUMBER;
    v_total NUMBER;
BEGIN
    v_salary := get_salary(emp_id);  -- Fetch salary
    v_bonus := calculate_bonus(emp_id);  -- Calculate bonus
    v_total := v_salary + v_bonus;  -- Compute total compensation
    RETURN v_total;
END total_compensation;
/
```

Call function

```
SELECT total_compensation(101) FROM dual;
```

Calling a Function Inside Another & Using a Nested Function (Local Function Inside Another Function)

- **A Function *get_bonus* calls another function *calculate_salary* inside it.**

```
CREATE OR REPLACE FUNCTION calculate_salary(emp_id NUMBER)
RETURN NUMBER IS
    v_salary NUMBER;
BEGIN
    SELECT salary INTO v_salary FROM employees WHERE employee_id =
emp_id;
    RETURN v_salary;
END calculate_salary;
/
```

```
CREATE OR REPLACE FUNCTION get_bonus(emp_id NUMBER) RETURN
NUMBER IS
    v_salary NUMBER;
    v_bonus NUMBER;
BEGIN
    -- Calling calculate_salary inside get_bonus
    v_salary := calculate_salary(emp_id);
    v_bonus := v_salary * 0.1; -- 10% bonus
    RETURN v_bonus;
END get_bonus;
/
```

Call our function

```
SELECT get_bonus(101) FROM dual;
```

- A function can contain a local function inside it, but the nested function is only accessible within the outer function.

```
CREATE OR REPLACE FUNCTION get_total_compensation(emp_id NUMBER) RETURN
NUMBER IS
    v_salary NUMBER;

    -- Nested function inside get_total_compensation
    FUNCTION get_allowance(emp_salary NUMBER) RETURN NUMBER IS
    BEGIN
        RETURN emp_salary * 0.2; -- 20% allowance
    END get_allowance;

BEGIN
    -- Fetch salary
    SELECT salary INTO v_salary FROM employees WHERE employee_id = emp_id;

    -- Call the nested function
    RETURN v_salary + get_allowance(v_salary);
END get_total_compensation;
/
```

Call the get_total_compensation

```
SELECT get_total_compensation(101) FROM dual;
```

Oracle Database SQL Functions Overview

Functions play a vital role in data processing and reporting within Oracle Database. Oracle provides a wide range of **in-built (predefined) SQL functions** that help manipulate and present data efficiently. These functions are used to perform operations on data stored in tables and return results in the desired format.

SQL*Plus

Categories of Oracle SQL Functions

Oracle SQL functions are broadly categorized into two types:

1. Single Row Functions

These functions operate on **individual rows** and return **one result per row**. They are often used in the SELECT, WHERE, and ORDER BY clauses to manipulate data as it is retrieved.

Character Functions: Used to manipulate string values.
Examples:

- **CONCAT**: Joins two strings
- **LENGTH**: Returns the number of characters in a string
- **UPPER**, **LOWER**: Convert case of string

Numeric Functions: Perform arithmetic and mathematical operations.
Examples:

- **ROUND**: Rounds a number
- **FLOOR**, **CEIL:** Returns the smallest or largest integer
- **MOD:** Returns the remainder of a division

Date Functions: Help manage and manipulate date and time values.
Examples:

- **SYSDATE:** Returns the current system date
- **MONTHS_BETWEEN**: Calculates the number of months between two dates
- **ADD_MONTHS, NEXT_DAY, LAST_DAY**

Conversion Functions: Convert one data type to another.
Examples:

- **TO_CHAR:** Converts a date or number to a string
- **TO_DATE:** Converts a string to a date
- **TO_NUMBER:** Converts a string to a number

2. Multiple Row Functions (Group or Aggregate Functions)

These functions operate on **a group of rows** and return **a single result** summarizing the group. They are commonly used with GROUP BY clauses in queries.

Common Aggregate Functions:

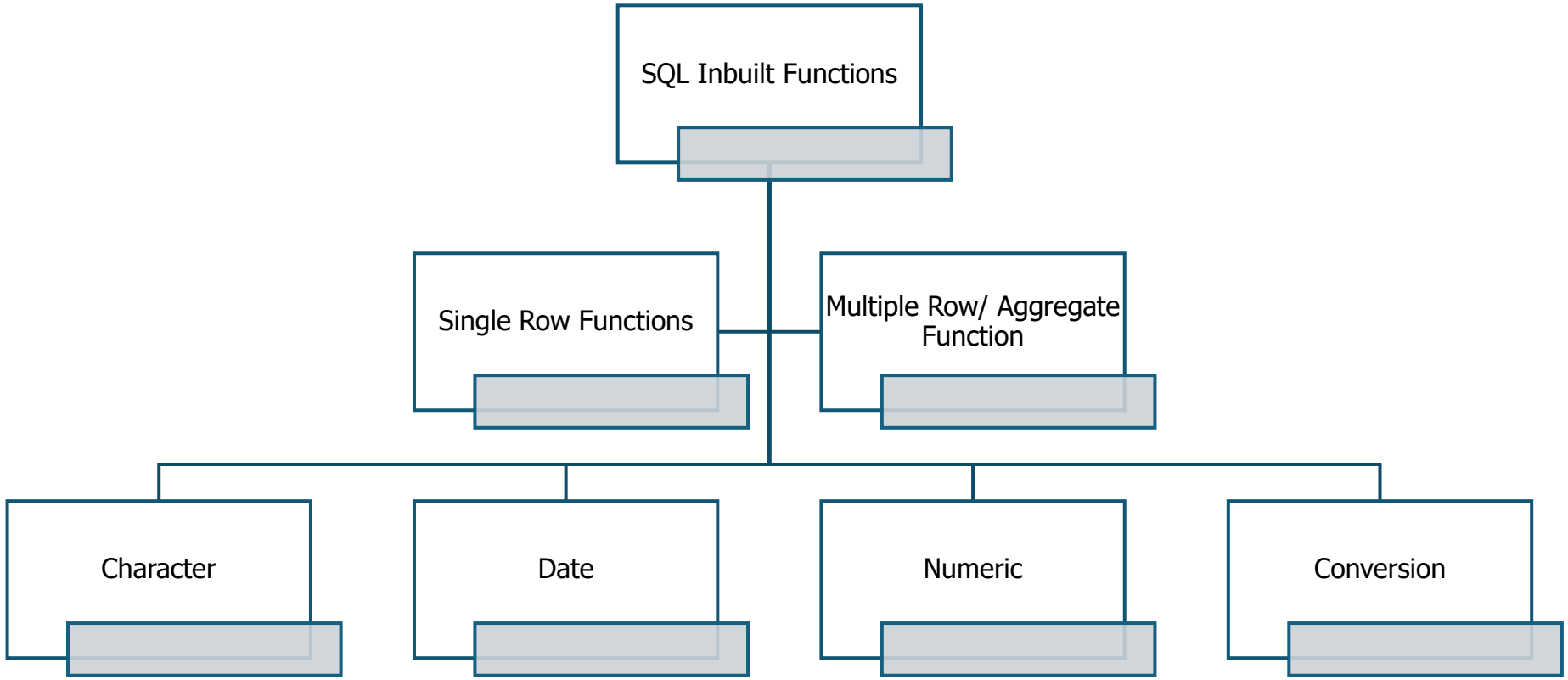
SUM
Returns the total sum of a numeric column.
Example: SUM(salary)

AVG
Calculates the average of values in a numeric column.
Example: AVG(marks)

COUNT
Returns the number of rows that meet a specified condition.
Example: COUNT(*) or COUNT(employee_id)

MAX / MIN
Return the highest or lowest value in a column.
Examples: MAX(price), MIN(hire_date)

Oracle Database SQL Functions Overview



Creating and Using Functions in Oracle SQL Database

Category	Description	Examples of Functions
Single-Row Functions	Functions that return a single result for each row processed.	
Numeric Functions	Perform calculations on numeric data.	- **ABS**: Returns the absolute value. - **ROUND**: Rounds a number to a specified decimal place. - **SQRT**: Returns the square root.
Character Functions	Operate on strings and return string values.	- **UPPER**: Converts a string to uppercase. - **SUBSTR**: Returns a substring from a string. - **CONCAT**: Combines two strings.
Datetime Functions	Work with date and time data types.	- **CURRENT_DATE**: Returns the current date and time. - **ADD_MONTHS**: Adds a specified number of months to a date. - **LAST_DAY**: Returns the last day of the month for a given date.
Conversion Functions	Convert values from one datatype to another.	**- TO_CHAR**: Converts a number or date to a string. - **TO_NUMBER**: Converts a string to a number. - **CAST**: Converts an expression from one datatype to another.
Miscellaneous Functions	Various functions that do not fit into the above categories.	- **NVL**: Returns a specified value if the expression is null. - **COALESCE**: Returns the first non-null expression. **- DECODE**: Performs conditional logic based on input.
Aggregate Functions	Return a single result based on a group of rows, typically used with GROUP BY.	- **SUM**: Returns the sum of values in a group. - **AVG**: Calculates the average of values in a group. - **COUNT**: Counts the number of rows in a group.
Analytic Functions	Similar to aggregate functions, but return multiple rows for each group.	- **RANK**: Assigns a rank to each row within a partition. - **LEAD**: Accesses data from the next row in the same result set. - **LAG**: Accesses data from the previous row in the same result set.

Oracle SQL Database – Numeric & Character Function

- **Numeric Functions**

ABS: Returns the absolute value of a number.

Syntax: ABS(number)

Example: SELECT ABS(-5) FROM dual; — **Output: 5**

ROUND: Rounds a number to a specified number of decimal places.

Syntax: ROUND(number, decimals)

Example: SELECT ROUND(45.678, 2) FROM dual; — **Output: 45.68**

SQRT: Returns the square root of a number.

Syntax: SQRT(number)

Example: SELECT SQRT(16) FROM dual; — **Output: 4**

- **Character Functions**

UPPER: Converts a string to uppercase.

Syntax: UPPER(string)

Example: SELECT UPPER('hello') FROM dual; — **Output: HELLO**

SUBSTR: Returns a substring from a string.

Syntax: SUBSTR(string, start, length)

Example: SELECT SUBSTR('Oracle', 2, 3) FROM dual; — **Output: rac**

CONCAT: Combines two strings into one.

Syntax: CONCAT(string1, string2)

Example: SELECT CONCAT('Oracle', 'PL/SQL') FROM dual; — **Output: OraclePL/SQL**

Oracle SQL Database – Datetime & Conversion Function

- **Datetime Functions**

CURRENT_DATE: Returns the current date and time of the session.

- **Syntax**: CURRENT_DATE
- **Example**: SELECT CURRENT_DATE FROM dual; — **Output: Current date without time**
- SELECT TO_CHAR(CURRENT_DATE, 'DD-MON-YY HH24:MI:SS') FROM dual;

ADD_MONTHS: Adds a specific number of months to a date.

- **Syntax**: ADD_MONTHS(date, number_of_months)
- **Example**: SELECT ADD_MONTHS(SYSDATE, 3) FROM dual; — **Output: Date after 3 months.**

LAST_DAY: Returns the last day of the month for a given date.

- **Syntax**: LAST_DAY(date)
- **Example**: SELECT LAST_DAY(SYSDATE) FROM dual; — **Output: Last day of the current month**.

- **Conversion Functions**

TO_CHAR: Converts a number or date to a string.

- **Syntax**: TO_CHAR(expression, format)
- **Example**: SELECT TO_CHAR(SYSDATE, 'DD-MON-YYYY') FROM dual; — **Output: 11-OCT-2024**

TO_NUMBER: Converts a string to a number.

- **Syntax**: TO_NUMBER(string)
- **Example**: SELECT TO_NUMBER('12345') FROM dual; — **Output: 12345**

CAST: Converts an expression from one datatype to another.

- **Syntax**: CAST(expression AS datatype)
- **Example**: SELECT CAST('12345' AS NUMBER) FROM dual; — **Output: 12345**

Oracle SQL Database – Miscellaneous & Aggregate Function

- **Miscellaneous Functions**

NVL: Replaces NULL with a specified value.

Syntax: NVL(expr1, expr2)

Example: SELECT NVL(NULL, 'default') FROM dual; — **Output: default**

COALESCE: Returns the first non-NULL expression.

Syntax: COALESCE(expr1, expr2, ..., exprN)

Example: SELECT COALESCE(NULL, NULL, 'value') FROM dual; — **Output: value**

DECODE: Performs conditional logic.

Syntax: DECODE(expr, search, result, default)

Example: SELECT DECODE(1, 1, 'One', 'Other') FROM dual; **— Output: One**

- **Aggregate Functions (Used with GROUP BY)**

SUM: Returns the sum of values in a group.

Syntax: SUM(expression)

Example: SELECT SUM(salary) FROM employees; **— Output: Sum of salaries.**

AVG: Calculates the average of values in a group.

Syntax: AVG(expression)

Example: SELECT AVG(salary) FROM employees; **— Output: Average salary.**

COUNT: Counts the number of rows in a group.

Syntax: COUNT(expression)

Example: SELECT COUNT(*) FROM employees; **— Output: Number of employees.**

Oracle SQL Database – Analytic

- **Analytic Functions**

SQL*Plus

RANK: Assigns a rank to each row within a partition.

Syntax: RANK() OVER (PARTITION BY expr ORDER BY expr)

Example: SELECT RANK() OVER (ORDER BY salary DESC) FROM employees; — **Ranks employees based on salary.**

LEAD: Accesses data from the next row in the result set.

Syntax: LEAD(expr, offset) OVER (ORDER BY expr)

Example: SELECT LEAD(salary, 1) OVER (ORDER BY salary) FROM employees; — **Retrieves the next salary in the sorted list**.

LAG: Accesses data from the previous row in the result set.

Syntax: LAG(expr, offset) OVER (ORDER BY expr)

Example: SELECT LAG(salary, 1) OVER (ORDER BY salary) FROM employees; — **Retrieves the previous salary in the sorted list.**

Built-in Functions in PL/SQL - Conversion Functions Example

PL/SQL provides various built-in functions to work with **strings** and **dates**. Below are commonly used functions along with their usage and examples.

These functions convert one data type to another.

Function	Description	Example	Output
TO_CHAR(value)	Converts a number or date to a character string.	TO_CHAR(123);	'123'
TO_DATE(string, format)	Converts a string to a date format. The string must match the specified format.	TO_DATE('2015-JAN-15', 'YYYY-MON-DD');	01/15/2015
TO_NUMBER(text, format)	Converts text into a number of the given format.	SELECT TO_NUMBER('1234','9999') FROM dual;	1234

Built-in Functions in PL/SQL - String Functions Example

These functions are used for manipulating character data.

Function	Description	Example	Output
INSTR(text, substring, start, occurrence)	Returns the position of a substring in a given text.	SELECT INSTR('AEROPLANE', 'E', 2, 2) FROM dual;	9
SUBSTR(text, start, length)	Extracts a substring from a given text.	SELECT SUBSTR('aeroplane', 1, 7) FROM dual;	'aeropla'
UPPER(text)	Converts text to uppercase.	SELECT UPPER('guru99') FROM dual;	'GURU99'
LOWER(text)	Converts text to lowercase.	SELECT LOWER('AerOpLane') FROM dual;	'aeroplane'
INITCAP(text)	Capitalizes the first letter of each word.	SELECT INITCAP('my story') FROM dual;	'My Story'
LENGTH(text)	Returns the number of characters in a string.	SELECT LENGTH('guru99') FROM dual;	6
LPAD(text, length, pad_char)	Pads the left side of a string with a specified character.	SELECT LPAD('guru99', 10, '$') FROM dual;	'$$$$guru99'
RPAD(text, length, pad_char)	Pads the right side of a string with a specified character.	SELECT RPAD('guru99', 10, '-') FROM dual;	'guru99----'
LTRIM(text)	Removes leading spaces from a string.	SELECT LTRIM(' Guru99') FROM dual;	'Guru99'
RTRIM(text)	Removes trailing spaces from a string.	SELECT RTRIM('Guru99 ') FROM dual;	'Guru99'

Built-in Functions in PL/SQL – Data Function Example

These functions help in date manipulation.

Function	Description	Example	Output
ADD_MONTHS(date, months)	Adds a specified number of months to a date.	SELECT ADD_MONTHS('2015-01-01', 5) FROM dual;	05/01/2015
SYSDATE	Returns the current date and time.	SELECT SYSDATE FROM dual;	10/4/2015 2:11:43 PM
TRUNC(date)	Rounds a date down to the nearest day.	SELECT TRUNC(SYSDATE) FROM dual;	10/4/2015
ROUND(date)	Rounds a date to the nearest day.	SELECT ROUND(SYSDATE) FROM dual;	10/5/2015
MONTHS_BETWEEN(date1, date2)	Returns the number of months between two dates.	SELECT MONTHS_BETWEEN(SYSDATE + 60, SYSDATE) FROM dual;	2

PL/SQL Function - Example

```
-- Function to Get Employee Full Name with User-Defined Exception Handling
CREATE OR REPLACE FUNCTION Get_Full_Name (
    p_First_Name IN VARCHAR2,
    p_Last_Name  IN VARCHAR2
) RETURN VARCHAR2
AS
    v_Full_Name VARCHAR2(100);

    -- Declare a user-defined exception
    empty_name_exception EXCEPTION;

BEGIN
    -- Raise an exception if either input is NULL
    IF p_First_Name IS NULL OR p_Last_Name IS NULL THEN
        RAISE empty_name_exception;
    END IF;

    -- Concatenate names
    v_Full_Name := p_First_Name || ' ' || p_Last_Name;

    RETURN v_Full_Name;

-- Exception handling section
EXCEPTION
    WHEN empty_name_exception THEN
        DBMS_OUTPUT.PUT_LINE('Error: First name or last name cannot be NULL.');
        RETURN NULL;

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unhandled error: ' || SQLERRM);
        RETURN NULL;
END;
/
```

Call the function

SQL*Plus

```
-- Anonymous block to test the function
DECLARE
    v_Full_Name VARCHAR2(100);
BEGIN
    -- Example call with a NULL first name to trigger user-defined exception
    v_Full_Name := Get_Full_Name(NULL, 'Smith');

    -- Output the result if successful
    IF v_Full_Name IS NOT NULL THEN
        DBMS_OUTPUT.PUT_LINE('Full Name: ' || v_Full_Name);
    END IF;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error in calling block: ' || SQLERRM);
END;
/
```

Output

```
Error: First name or last name cannot be NULL.

PL/SQL procedure successfully completed.
```

Problem Statement: Create a Function to Compute Total Allowances by Role

Your organization stores employee allowance details in a database table named Allowances. Each record in the table corresponds to an individual allowance associated with a specific Role_ID.

SQL*Plus

Table Structure: Allowances

Column Name	Data Type	Description
Allowance_ID	NUMBER	Unique identifier for each allowance
Role_ID	NUMBER	The ID representing the employee role
Allowance_Amount	NUMBER	The amount of the allowance

Task:
Write a PL/SQL function named Get_Total_Allowance that:

- Accepts a Role_ID as an input parameter.
- Calculates and returns the **total allowance amount** associated with that role.
- If no allowances exist for the given role, return **0**.
- If an error occurs (e.g., due to unexpected issues), handle it using exception handling and return **-1** to indicate failure.

Requirements:

1. Use SELECT SUM(Allowance_Amount) to compute the total.
2. Use the NVL function to handle NULL totals.
3. Include exception handling using WHEN OTHERS.

Solution : Create a Function to Compute Total Allowances by Role

```
-- Create or replace the function to get total allowance by Role_ID
CREATE OR REPLACE FUNCTION Get_Total_Allowance (
    p_Role_ID NUMBER  -- Input parameter for Role_ID
)
RETURN NUMBER  -- Return type of the function
AS
    v_Total_Allowance NUMBER := 0;  -- Variable to store total allowance
BEGIN
    -- Compute the sum of applicable allowances for the given Role_ID
    SELECT NVL(SUM(Allowance_Amount), 0)
    INTO v_Total_Allowance
    FROM Allowances
    WHERE Role_ID = p_Role_ID
      AND is_applicable = 'Y';  -- Only include applicable allowances

    -- Return the result
    RETURN v_Total_Allowance;

-- Exception handling for unexpected errors
EXCEPTION
    WHEN OTHERS THEN
        -- Log or handle the error if needed
        RETURN -1;  -- Return -1 to indicate an error
END Get_Total_Allowance;
/
```

SQL*Plus

```
-- Call the Get_Total_Allowance function
DECLARE
    v_Role_ID NUMBER := 101;  -- Example Role_ID
    v_Total NUMBER;
BEGIN
    v_Total := Get_Total_Allowance(v_Role_ID);

    IF v_Total = -1 THEN
        DBMS_OUTPUT.PUT_LINE('Error occurred while fetching
allowance.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Total Allowance for Role_ID ' ||
v_Role_ID || ': ' || v_Total);
    END IF;
END;
/
```

Assignment: Exploring SQL Window Functions – Instructions & Objectives

Total Marks: 10

Important Instructions:

- This assignment is for **two students** working as a team.
- Identify **one classmate** to collaborate with and form a **group**.
- Create a **group repository on GitHub** where both members will contribute.
- **Add me as a collaborator** so I can review your work (GitHub username: **ericmaniraguha**).
- Choose a **funny group name** for your repository (e.g ThePrimaryKeys, Commit_the_Query etc.).
- I will **track each student's contributions** on GitHub. **Collaboration is key!**
- **Zero contribution = Zero marks**, so ensure that **both members push code** to the repository.

Objective:

Students will:

- Work with **SQL Window Functions** on a **generic dataset**.
- Perform various analytical queries using **LAG(), LEAD(), RANK(), DENSE_RANK(), ROW_NUMBER(), and aggregate functions**.
- Explain their **queries, results, and real-life applications** of window functions.
- **Document their work properly** and push everything to GitHub.

Assignment: Exploring SQL Window Functions - Tasks & Queries (10 Points)

Instructions:

- Choose or **create a dataset** related to any domain (**employees, sales, students, orders, products, etc.**).
- Perform **ALL** of the following queries and provide explanations for each.

1. Compare Values with Previous or Next Records

- Use **LAG()** and **LEAD()** functions to compare a chosen column (**salary, sales, scores, prices, etc.**) with the previous and next records.
- Display whether the value is **HIGHER, LOWER, or EQUAL** compared to the previous record.

2. Ranking Data within a Category

- Use **RANK()** and **DENSE_RANK()** to rank records within a category (**e.g., employees by department, students by class, products by category**).
- Explain the difference between **RANK()** and **DENSE_RANK()** with examples.

3. Identifying Top Records

- Fetch the **top 3 records** from each category (**e.g., top 3 highest salaries per department, top 3 best-selling products per region**).
- Ensure duplicate values are **handled appropriately** using ranking functions.

4. Finding the Earliest Records

- Retrieve the **first 2 records** from each category based on a **date column** (**e.g., first employees to join, first sales transactions, first registered students**).
- Explain the logic behind your query.

5. Aggregation with Window Functions

- Select **all records** and calculate:
  - The **maximum value** within each category.
  - The **overall maximum value** across all records.
- Use **PARTITION BY** to differentiate between **category-level** and **overall calculations**.

Assignment: Exploring SQL Window Functions - Submission Guidelines

Submission Guidelines:

1. **Create a GitHub repository** for your group work.
2. **Push the following files** to the repository:
    - **SQL scripts** (table creation, data insertion, queries).
    - **README.md** (explanations, screenshots, findings, real-life applications).

3. **Add me as a collaborator** to your GitHub repository for assessment.
4. **Choose a funny group name** for your repository.
5. **Submit your GitHub repository link** before the deadline.
    - **Deadline:** April 17, 2025, 11h59 pm
    - **Submission: GitHub Repository Link**

You can watch this YouTube tutorial on SQL Window Functions: Click Here.

Next Step: A Simple Quiz
After completing this assignment, you will take a **short quiz** to test your understanding of **window functions**.

1 Thessalonians 5:11
"Therefore encourage one another and build each other up, just as in fact you are doing."

PL/SQL Using Aggregate Functions (SUM, AVG) in a Procedure

Code with Exception Handling

```
CREATE OR REPLACE PROCEDURE get_salary_stats (
    total_sal OUT NUMBER,
    avg_sal OUT NUMBER
) AS
BEGIN
    -- Initialize output variables
    total_sal := 0;
    avg_sal := 0;

    BEGIN
        -- Use SELECT INTO to assign aggregate results to variables
        SELECT SUM(salary), AVG(salary)
        INTO total_sal, avg_sal
        FROM employees;

        -- Optionally, display the results
        DBMS_OUTPUT.PUT_LINE('Total Salary: ' || total_sal);
        DBMS_OUTPUT.PUT_LINE('Average Salary: ' || avg_sal);

    EXCEPTION
        -- Handle case when the employees table is empty or if there is any other error
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('No data found in employees table.');
            total_sal := 0;
            avg_sal := 0;

        WHEN OTHERS THEN
            -- Catch any other exceptions and log the error message
            DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
            total_sal := NULL;
            avg_sal := NULL;
    END;
END;
```

SQL*Plus

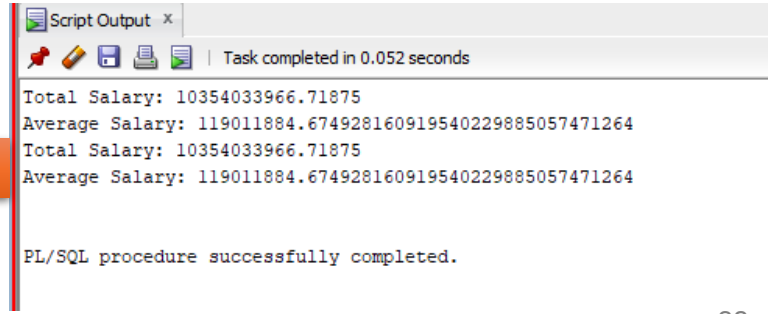
Call this function as follows

```
SET SERVEROUTPUT ON;

DECLARE
    v_total_salary NUMBER;
    v_avg_salary NUMBER;
BEGIN
    -- Call the procedure and pass the variables to hold the OUT parameters
    get_salary_stats(v_total_salary, v_avg_salary);

    -- Optionally display the values in the anonymous block
    DBMS_OUTPUT.PUT_LINE('Total Salary: ' || v_total_salary);
    DBMS_OUTPUT.PUT_LINE('Average Salary: ' || v_avg_salary);
END;
/
```

Output



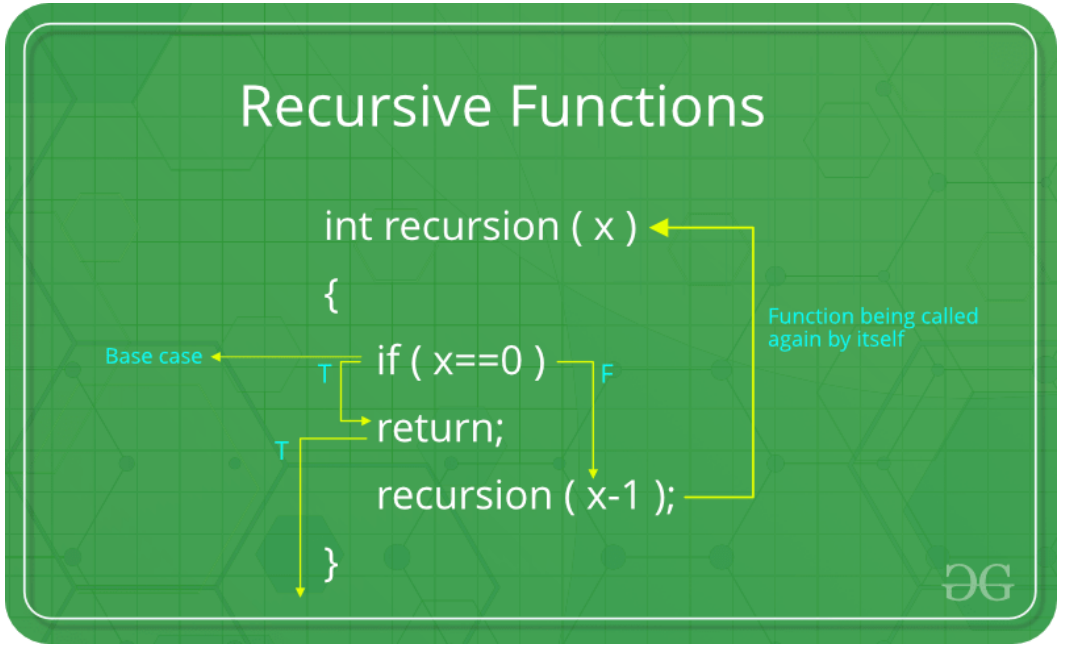
PL/SQL Recursive Functions

Introduction to Recursive Functions

A **recursive function** solves a problem by calling itself with smaller instances of the same problem. It is particularly useful for problems that can be broken down into similar subproblems, resulting in more compact and readable code.

SQL*Plus

Key Uses and Benefits	**Solving Complex Tasks**: Recursive functions break down complex problems into simpler subproblems, leading to concise and readable code.
	Divide and Conquer: Used in algorithms like **merge sort** and **quicksort**, recursion divides a problem into smaller parts, solves them, and combines the results.
	Backtracking: Ideal for problems like **N-Queens** and **Sudoku**, where the solution requires exploring multiple possibilities and undoing decisions.
	Dynamic Programming: Recursion solves overlapping subproblems efficiently, as seen in problems like the **Fibonacci sequence** or **knapsack problem**.
	Tree and Graph Traversal: Recursion simplifies the exploration of hierarchical structures like trees and graphs, making tasks like **depth-first search (DFS)** easier to implement.



Characteristics of Recursive Functions

Base Case:

The base case is a condition that terminates the recursion. It is essential to prevent the function from calling itself indefinitely, which could lead to a **stack overflow** or infinite recursion. Without a base case, recursion would continue forever.

Recursive Call:

The function calls itself with modified arguments. This step reduces the problem's size and brings it closer to the base case. The recursive call is the core of recursion, as it breaks down the problem into smaller instances.

Termination Condition:

The recursive calls must eventually reach the base case. This ensures the function will stop calling itself and begin returning values back through the recursive calls, ultimately solving the problem.

Conclusion

Recursive functions provide an elegant, efficient solution to problems that can be divided into smaller, similar subproblems, making them essential in tasks such as sorting, backtracking, dynamic programming, and tree traversal.

```
-- Recursive Function to calculate Factorial of a number in PL/SQL

CREATE OR REPLACE FUNCTION factorial(n IN NUMBER) RETURN NUMBER IS
BEGIN
-- Base case: if x is 0, return 1 (because 0! = 1)
    IF n = 0 THEN
        RETURN 1;
    ELSE
-- Recursive case: x * factorial of (x-1)
        RETURN n * factorial(n - 1);
    END IF;
END factorial;
/
```

SQL*Plus

```
-- Driver Code
DECLARE
    n NUMBER := 4;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Factorial of ' || n || ' is: ' || factorial(n));
END;
/
```

How It Works:

- **Base Case**: If the input n is 0, the function returns 1, as 0! = 1.
- **Recursive Case**: If n > 0, the function multiplies n by the factorial of n-1, i.e., f = n * factorial_func(n-1).

This process continues until the base case (n = 0) is reached, at which point the function stops recursing and returns the final result.

Characteristics of Recursive Functions Fibonacci

```
-- Recursive Fibonacci Function without OUT parameter
CREATE OR REPLACE FUNCTION fibonacci_func(n IN NUMBER) RETURN NUMBER IS
    f NUMBER;  -- Local variable to hold the Fibonacci result

BEGIN
    -- Exception handling for invalid input
    IF n < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Input must be a non-negative integer');
    END IF;

    -- Base case: F(0) = 0 and F(1) = 1
    IF n = 0 THEN
        f := 0;
    ELSIF n = 1 THEN
        f := 1;
    ELSE
        -- Recursive case: F(n) = F(n-1) + F(n-2)
        f := fibonacci_func(n - 1) + fibonacci_func(n - 2);
    END IF;

    -- Return the computed Fibonacci value
    RETURN f;

EXCEPTION
    -- Exception handler for other errors
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        RETURN NULL;
END fibonacci_func;
/
```

The **Fibonacci sequence** is a series of numbers where each number is the sum of the two preceding ones. The sequence typically starts with 0 and 1. So, the first few numbers in the Fibonacci sequence are:

SQL*Plus

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Formula:

$$F(0) = 0$$
$$F(1) = 1$$
$$F(n) = F(n-1) + F(n-2) \quad \text{for } n > 1$$

```
-- Driver Code (Anonymous Block) to Call the Function
DECLARE
    num NUMBER := 7;          -- Input number for Fibonacci calculation
    fibonacci_result NUMBER;  -- Variable to store the result
BEGIN
    -- Call the function and assign the result to fibonacci_result
    fibonacci_result := fibonacci_func(num);

    -- Output the result using DBMS_OUTPUT
    DBMS_OUTPUT.PUT_LINE('Fibonacci of ' || num || ' is: ' || fibonacci_result);
END;
/
```

PL/SQL Procedure – Triangle Patterns

Create Function

```
CREATE OR REPLACE PROCEDURE print_triangle_pattern (n IN NUMBER, stars_count
OUT NUMBER) IS
BEGIN
  stars_count := 0; -- Initialize stars count
  FOR i IN 1..n LOOP
    -- Loop through each row
    FOR j IN 1..i LOOP
      -- Print stars on the current row
      DBMS_OUTPUT.PUT(' * ');
      stars_count := stars_count + 1; -- Increment stars count
    END LOOP;
    -- Move to the next line after each row
    DBMS_OUTPUT.PUT_LINE('');
  END LOOP;

  -- Print the total stars printed
  DBMS_OUTPUT.PUT_LINE('Total stars printed: ' || stars_count);
END print_triangle_pattern;
/
```

SQL*Plus

Calling our function

```
DECLARE
  total_stars NUMBER; -- Variable to store the total stars printed
BEGIN
  -- Call the procedure with n=5 and get the total stars printed
  print_triangle_pattern(5, total_stars);
  -- Print the total stars printed
  DBMS_OUTPUT.PUT_LINE('Total stars printed: ' || total_stars);
END;
/
```

Output

```
*
 * *
 * * *
 * * * *
 * * * * *
Total stars printed: 15
Total stars printed: 15
```

PL/SQL Procedure – Triangle Patterns

Create Function

```
CREATE OR REPLACE PROCEDURE print_pyramid_pattern (n IN NUMBER) IS
BEGIN
  -- Check for valid input (n must be positive)
  IF n <= 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Error: The number of rows must be a positive integer.');
  END IF;

  -- Loop to print the pyramid pattern
  FOR i IN 1..n LOOP -- Loop for rows – outer loop
    FOR j IN 1..n-i LOOP -- Loop for spaces –inner loop
      DBMS_OUTPUT.PUT(' ');
    END LOOP;

    FOR k IN 1..2*i-1 LOOP -- Loop for stars – inner loop
      DBMS_OUTPUT.PUT('*');
    END LOOP;

    DBMS_OUTPUT.NEW_LINE; -- Move to next row
  END LOOP;

EXCEPTION
  -- Handle specific exceptions (e.g., when the input is invalid or there are database-related errors)
  WHEN VALUE_ERROR THEN
    DBMS_OUTPUT.PUT_LINE('Error: Invalid value encountered during execution.');
  WHEN OTHERS THEN
    -- This block captures any other errors
    DBMS_OUTPUT.PUT_LINE('Unexpected error occurred: ' || SQLERRM);
END print_pyramid_pattern;
/
```

Adds leading spaces to center-align the pyramid.

Runs from 1 to n-i, where:

- i = 1 → Prints n-1 spaces.
- i = 2 → Prints n-2 spaces.
- i = n → Prints 0 spaces

Prints **stars (*)** in each row.
The number of stars in row i is 2*i - 1, forming the pyramid shape:

- Row 1: 1 star (2*1 - 1 = 1).
- Row 2: 3 stars (2*2 - 1 = 3).
- Row 3: 5 stars (2*3 - 1 = 5).
- Row n: (2*n - 1) stars.

Calling our function

```
BEGIN
  -- Call the procedure with n=5 to print a pyramid pattern
  print_pyramid_pattern(5);
END;
/
```

Output

```
    *
   ***
  *****
 *******
*********
```

PL/SQL Programming Task: Adjusting Employee Salaries (1/2)

I. Understanding Parameters

1. Task:
Create a **stored procedure** named adjust_employee_salary that includes the following parameters:

- An **IN** parameter emp_id (the employee's ID).
- An **INOUT** parameter salary (the current salary of the employee).
- An **OUT** parameter new_salary (the updated salary after adjustment).

Requirements:

- The procedure must use the current salary value to calculate the adjusted salary.
- The new_salary must reflect the updated amount based on defined logic.
- Include proper **exception handling** (e.g., handle the case where the employee ID does not exist or invalid salary is provided).

Follow-up Question:
What will happen if the salary (INOUT parameter) is not assigned a value before the procedure ends? Explain the impact.

SQL*Plus

II. Using Functions

2. Task:
Create a **function** named calculate_bonus that:

- Accepts a **NUMBER** parameter (the employee's salary).
- Returns a **NUMBER** representing the calculated bonus (e.g., 10% of the salary).

Requirements:

- The function must return salary * 0.10 as the bonus amount.
- Include basic **exception handling** inside the function to manage invalid or null salary inputs.

Follow-up Question:
How can you call the calculate_bonus function from within your adjust_employee_salary procedure?
Write the relevant code snippet showing the function call.

PL/SQL Programming Task: Adjusting Employee Salaries (2/2)

III. Implementing Logic

3. Task:
Inside the adjust_employee_salary procedure, implement the following logic:

- If the **salary is below $50,000**, increase it by the calculated bonus.
- If the **salary is $50,000 or above**, increase it by a fixed amount of **$5,000**.
- If the **adjusted salary exceeds $100,000**, **cap it at $100,000** and notify via output message.

Requirements:

- Use conditional (IF...THEN...ELSE) logic.
- Implement exception handling for any unexpected errors (e.g., OTHERS exception).
- Ensure that the new_salary OUT parameter reflects the final adjusted value after applying all business rules.

Follow-up Question:
How would you implement a condition to ensure that the adjusted salary does not exceed $100,000?
Provide the **conditional statement** used to enforce this rule.

SQL*Plus

Conclusion: Key Points About PL/SQL Functions

SQL*Plus

Must Return a Value

- [ ] Functions always return a value (scalar, table, or collection).

Improve Performance

- [ ] Executed on the server, reducing data transfer and boosting efficiency.

Enhance Security

- [ ] Encapsulate logic and control access to sensitive operations or data.

Simplify Logic

- [ ] Break down complex tasks into reusable, manageable components.

Handle Exceptions

- [ ] Use EXCEPTION blocks to gracefully manage errors.

Simplify Data Access

- [ ] Encapsulate complex queries (e.g., joins) for cleaner, reusable access patterns.

Keep Learning!